

Documentazione Ospedale - Gruppo 06

Montella S. - Lerro C.P. - Mastellone L.

Sommario

1	Introduzione	2
2	Struttura dei package	2
3	Il package model	2
3.1	Gestione degli attori	2
3.2	Struttura dell'Ospedale	3
3.3	Processi burocratici	3
4	Il Package Controller	3
5	Il package <i>GUI</i>	3
5.1	Autenticazione e Finestre (Utente e Amministratore)	4
5.2	Schermate dedicate alle singole risorse	4
6	Il package <i>DAO</i>	4
7	Il package <i>implementazioneDao</i>	5
8	Il package <i>database_connection</i>	6
8.1	Gestione delle transazioni	6

1 Introduzione

Il presente documento ha la finalità di descrivere l'architettura software dell'applicazione deputata alla Gestione di un'azienda Ospedaliera. L'applicazione è stata costruita seguendo il concetto di architettura DAO (Data Access Object) che prevede la separazione dei vari stadi logici presenti, i quali sono: logica di dominio, flusso di controllo e GUI.

2 Struttura dei package

I package forniscono la possibilità di contenere tutte le classi che fanno parte della stessa "Macroarea" di progettazione, in un unico punto. I package descritti nel seguente documento sono:

- *model*: Contiene le entità del dominio del problema.
- *controller*: Gestisce le interazioni tra i vari package dell'applicazione e regola le azioni che conseguono ad eventuali input provenienti dall'esterno.
- *GUI*: Contiene le classi dedicate all'Interfaccia Grafica.
- *DAO*: Definisce le interfacce contenenti i metodi astratti che verranno poi implementati, permettendo l'accesso ai dati.
- *implementazioneDAO*: Contiene le implementazioni delle interfacce DAO precedentemente citate. In questo package si implementa una particolare API di Java, chiamata JDBC che permette all'utente di effettuare operazioni CRUD all'interno di un DB, attraverso query SQL.
- *database_connection*: Gestisce tutte le operazioni che prevedono l'interazione verso la base di dati, come la connessione al DB, la configurazione e il rilascio delle query JDBC.

3 Il package model

Il package *model* contiene e implementa le entità del problema.

3.1 Gestione degli attori

- *Utente*: La sottoclasse *Utente* rappresenta un generico utente che accede al sistema informativo. Possiede i seguenti attributi: login, matricola, nome, cognome.
- *Medico*: La sottoclasse *Medico* oltre ad ereditare gli attributi della sua classe padre, è dotata di attributi capaci di identificare il ruolo dell'utente che accede al sistema. I quali sono: iscrizioneAlbo, specializzazione, repartoDiAppartenenza.

- Amministratore: La sottoclasse Amministratore, come Medico, è figlia di Utente. Non implementa attributi supplementari a quelli ereditati, poichè questi ultimi la completano
- Paziente: Rappresenta il paziente (che può essere in degenza o meno). Gli attributi che lo caratterizzano sono: nome, cognome, cf, dataNascita, sesso, residenza, diagnosi.

3.2 Struttura dell'Ospedale

Le seguenti classi definiscono fisicamente la struttura dell'Ospedale:

- Reparto: Rappresenta il singolo reparto ospedaliero.
- Stanza: Rappresenta l'unità medica contenente al suo interno dei Letti.
- Letto: La classe Letto rappresenta il singolo posto letto, assegnato ad un Paziente ricoverato.

3.3 Processi burocratici

Le seguenti classi definiscono le componenti dei processi burocratici dell'azienda ospedaliera.

- TurnoLavoro: La classe TurnoLavorativo rappresenta il lasso di tempo, nel quale viene richiesta la presenza all'interno dell'Ospedale ad un Medico.
- Agenda: la classe Agenda definisce uno strumento in possesso di medici, per tenere traccia delle loro prestazioni e dei loro turni lavorativi.
- Prestazione: La classe Prestazione identifica una prestazione di tipo medico, fornita da parte di un Medico.
- Assenza: La classe Assenza identifica un periodo, nel quale un Medico è impossibilitato a coprire i turni di lavoro ad esso assegnati.

4 Il Package Controller

Il package *controller* ha il compito di mediare e gestire le richieste provenienti dalla GUI, dal model e dal livello DAO che assicura la persistenza dei dati nel DB. Un esempio delle operazioni gestite dal Controller, potrebbe essere quello dell'autenticazione iniziale del singolo utente.

5 Il package GUI

Il package GUI contiene l'interfaccia grafica che sarà poi utilizzata dall'utente per comunicare con l'applicazione.

5.1 Autenticazione e Finestre (Utente e Amministratore)

- *Registrazione*: La schermata di registrazione permette all'utente di prepararsi a fare l'uso della schermata di **Login**, inserendo i propri dati, anche in base al ruolo che si ha all'interno dell'Ospedale.
- *Login*: La schermata di avvio dell'applicazione. Permette agli utenti di effettuare l'autenticazione ed accedere al sistema.
- *SchermataAmministratore*: Finestra dedicata agli utenti di tipo Amministratore, dal quale è possibile avere il totale controllo dell'intera struttura Ospedaliera e delle sue componenti.
- *SchermataMedico*: Finestra dedicata agli utenti di tipo Medico. Funge da area riservata, dove ogni medico può controllare le risorse interne dell'ospedale e le pratiche di sua competenza.

5.2 Schermate dedicate alle singole risorse

- *Prestazioni*: Permette di controllare le prestazioni mediche erogate o programmate internamente all'Ospedale.
- *Turni*: Permette di programmare i turni dei medici.
- *Pazienti*: Qui sono contenute tutte le informazioni sui singoli pazienti che sono attualmente presenti all'interno dell'Ospedale o che sono stati presenti all'interno di esso.
- *Ricovero*: Offre la possibilità di ricoverare un paziente nella struttura, inserendo le sue informazioni all'interno del sistema.
- *Medici*: Questa schermata mostra tutte le informazioni sui medici presenti all'interno dell'Ospedale, è riservata agli amministratori.
- *Letti*: Qui vengono mostrati tutti i Letti presenti nei vari reparti dell'Ospedale.
- *Dimissioni*: Permette di gestire le dimissioni dei pazienti ricoverati.
- *Calendario*: Permette ai medici di tenere traccia dei loro impegni attraverso questa schermata.

6 Il package *DAO*

Il package DAO permette di definire il livello logico dedicato ai dati. Costituito da interfacce quest'ultimo non si preoccupa dell'implementazione nel DB.

- *Amministratore.DAO*: Interfaccia che permette di gestire l'autenticazione, la registrazione e l'aggiornamento delle informazioni degli utenti *Amministratore*.

- *Medico_DAO*: Analogamente alla precedente, questa interfaccia permette di effettuare operazioni di autenticazione e registrazione dedicate agli utenti *Medico*. Inoltre gestisce le operazioni di eliminazione, aggiornamento o recupero delle informazioni legate agli utenti *Medico*.
- *PazienteDAO*: Simile alle precedenti, permette di effettuare le medesime operazioni ma sui *Pazienti*.
- *LettoDAO*: Regola le operazioni di gestione dei singoli letti.
- *RicoveroDAO*: Definisce le operazioni di gestione dei ricoveri.
- *PrestazioneDAO*: Interfaccia deprecata all'inserimento, aggiornamento ed eliminazione delle prestazioni mediche.
- *AssenzaDAO*: Istanza i metodi astratti che verranno poi sfruttati per la gestione delle assenze.
- *DimissioniDAO*: Definisce le operazioni necessarie alla gestione delle *Dimissioni*.
- *AgendaDAO*: Definisce i metodi astratti che regoleranno il funzionamento delle agende mediche.

7 Il package *implementazioneDao*

In questo package vi sono contenute le implementazioni delle interfacce astratte precedentemente discusse, definite nel package *dao*. In particolare, nelle classi che fanno parte del package *implementazioneDAO* interagiscono con un DB (nel nostro caso si è usato PostgreSQL), attraverso le conseguenti query SQL, che variano in base all'operazione da effettuare.

- *AmministratorePostgresDAO*: Implementa *AmministratoreDAO*. Gestisce operazioni di INSERT, UPDATE e DELETE sulla tabella *Amministratore*.
- *PazientePostgresDAO*: Implementa *PazienteDAO*. Effettua query di INSERT, UPDATE, DELETE sulla tabella *Paziente*.
- *MedicoPostgresDAO*: Implementa l'interfaccia *MedicoDAO*. Analogamente alla precedente implementazione, esegue operazioni di INSERT, UPDATE, DELETE sulla tabella *Medico*.
- *RicoveroPostgresDAO*: Implementa l'interfaccia *RicoveroDAO*. Trasforma i dati inerenti ad un ricovero (fra i quali i dati del paziente e del letto assegnato) in query SQL per inserirli all'interno del DB.
- *TurnoLavoroPostgresDao*: Implementa *TurnoLavoroDAO*. Attraverso query SQL permette di gestire i turni di lavoro dell'intero Ospedale.

- *LettoPostgresDAO*: Implementa l'interfaccia *LettoDAO*. Permette di gestire le operazioni sui Letti, sempre attraverso query SQL.
- *DimissioniPostgresDAO*: Implementa *DimissioniDAO*. Offre la possibilità di gestire le operazioni da effettuare sulla tabella Dimissioni.
- *AssenzaPostgresDAO*: Implementa *AssenzaDAO*. Gestisce le operazioni relative ad un'eventuale richiesta di assenza.
- *PrestazioniPostgresDAO*: Implementa *PrestazioniDAO*. Si occupa di gestire tramite query SQL le prestazioni erogate dall'Ospedale.
- *AgendaPostgresDAO*: Implementa *AgendaDAO*. Attraverso questa implementazione, si permette ai medici e agli amministratori di gestire gli eventi legati all'Agenda. L'Agenda possiede degli eventi, prelevati dalla tabella Prestazioni.

8 Il package `database_connection`

Il package `database_connection` gestisce il livello più basso dell'infrastruttura dati, occupandosi dell'apertura e della chiusura sicura delle connessioni verso il DBMS.

- `ConnessioneDatabase`: Classe (implementata secondo il pattern Singleton) che si occupa di istanziare e mantenere un'unica connessione JDBC (Java Database Connectivity) condivisa verso PostgreSQL, tramite il connection pooler offerto da Supabase. La connessione viene riutilizzata per tutta la durata dell'applicazione e non viene mai chiusa esplicitamente da questa classe: sono le singole implementazioni DAO a essere responsabili della chiusura dei propri Statement e ResultSet tramite blocchi try-with-resources, per evitare memory leak senza invalidare la connessione condivisa. La classe fornisce inoltre un meccanismo per eseguire più operazioni come un'unica transazione atomica (vedi paragrafo 8.1), garantendo la coerenza dei dati quando un'azione utente richiede più scritture correlate sul database.

8.1 Gestione delle transazioni

Alcune operazioni applicative richiedono che più scritture sul database avvengano in modo atomico: ad esempio, la registrazione di un ricovero comporta sia l'aggiornamento dello stato di un letto (da libero a occupato) sia l'inserimento del nuovo ricovero, mentre una dimissione comporta sia la chiusura del ricovero sia la liberazione del letto. Se una delle due operazioni fallisse dopo che l'altra è già stata confermata, il database si troverebbe in uno stato incoerente (un letto bloccato senza ricovero, o viceversa). Per evitare questo scenario, `ConnessioneDatabase` espone un metodo che disattiva temporaneamente l'autocommit della

connessione, esegue le operazioni richieste, e conferma (commit) o annulla (roll-back) entrambe insieme in base al loro esito complessivo, ripristinando infine il comportamento normale della connessione.